

JavaScript Throttling & Debouncing

Beginner → Intermediate → Advanced Interview & Reference Notes

Core topics: high-frequency browser events, throttling, debouncing, closures, timers, timestamps, event arguments, leading/trailing execution, cancellation, practical use cases, and interview questions.

1. The Problem: High-Frequency Events

Some browser events can fire many times in a short period. Examples include **scroll**, **resize**, **mousemove**, **input**, and drag-related events.

If the event handler performs expensive work on every event, it can cause unnecessary CPU usage, excessive DOM work, too many API requests, and poor responsiveness.

```
window.addEventListener("scroll", handleScroll);
```

The browser may invoke `handleScroll` repeatedly while the user is scrolling. Throttling and debouncing are techniques used to control this execution.

2. Throttling

Definition: Throttling limits a function so that it executes **at most once during a specified time interval**, even when the associated event occurs repeatedly.

If the delay is 2000 ms and the event continues continuously, the function can execute roughly once every 2 seconds.

```
Event: ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
Function: ✓ ✓ ✓
|-----|-----|
2s 2s
```

3. Basic Time-Based Throttling

```
let time = 0;

function handleScroll() {
  let currentTime = Date.now();

  if (currentTime - time >= 2000) {
    console.log("scrolling");
    time = currentTime;
  }
}

window.addEventListener("scroll", handleScroll);
```

The variable `time` stores the timestamp of the last permitted execution. Each event compares the current timestamp with that stored timestamp.

The condition `currentTime - time >= delay` determines whether enough time has elapsed.

4. Why the Timestamp Check Works

Assume the last execution occurred at 10,000 ms and the delay is 2,000 ms.

Current time	Elapsed time	Execute?
10,500 ms	500 ms	No

11,000 ms	1,000 ms	No
12,000 ms	2,000 ms	Yes
13,000 ms	1,000 ms	No
14,000 ms	2,000 ms	Yes

5. Reusable Throttle Function

```
function throttle(callback, delay) {
  let time = 0;

  return function () {
    let currentTime = Date.now();

    if (currentTime - time >= delay) {
      callback();
      time = currentTime;
    }
  };
}
```

The function receives the callback and delay once, then returns a new function. That returned function is registered as the event handler.

```
function scrollHandler() {
  console.log("Scrolling");
}

let throttleHandler = throttle(scrollHandler, 2000);

window.addEventListener("scroll", throttleHandler);
```

The same throttle function can be reused for resize, mouse movement, scrolling, and other high-frequency events.

6. Closure in Throttling

The returned function can continue to access time, callback, and delay after throttle() has finished executing. This is a closure.

```
function throttle(callback, delay) {
  let time = 0;

  return function () {
    // time is preserved between calls
    ...
  };
}
```

Each call to throttle() creates its own closure and therefore its own independent time value.

7. Passing Event Arguments

A reusable throttle should normally forward the event object or other arguments to the callback.

```
function throttle(callback, delay) {
  let time = 0;

  return function (event) {
    let currentTime = Date.now();

    if (currentTime - time >= delay) {
      callback(event);
      time = currentTime;
    }
  };
}
```

```

}
};
}

function handleScroll(event) {
  console.log(event);
}

let handler = throttle(handleScroll, 2000);
window.addEventListener("scroll", handler);

```

8. Debouncing

Definition: Debouncing delays function execution until a specified amount of time has passed **without the event occurring again**.

Every new event resets the timer. Therefore, the callback executes only after the events stop for the specified delay.

```

Typing:
H → He → Hel → Hell → Hello
↓
Each input resets the timer
↓
No input for 2 seconds
↓
Callback executes

```

9. Basic Debouncing

```

let timerId;

function searchHandler(event) {
  clearTimeout(timerId);

  timerId = setTimeout(() => {
    console.log(
      "API call for search suggestion",
      event.target.value
    );
  }, 2000);
}

search.addEventListener("input", searchHandler);

```

`clearTimeout(timerId)` cancels the previously scheduled callback. A new `setTimeout()` is then created. If another input event occurs before 2 seconds, the previous timer is cancelled again.

10. Reusable Debounce Function

```

function debounce(callback, delay) {
  let timerId;

  return function (event) {
    clearTimeout(timerId);

    timerId = setTimeout(() => {
      callback(event);
    }, delay);
  };
}

function searchHandler(event) {
  console.log(
    "API call for search suggestion:",

```

```
event.target.value
);
}

let debounceHandler = debounce(searchHandler, 2000);

search.addEventListener("input", debounceHandler);
```

11. Closure in Debouncing

The timerId variable must survive between calls because each new event needs access to the previous timer.

```
function debounce(callback, delay) {
  let timerId;

  return function (event) {
    clearTimeout(timerId);
    timerId = setTimeout(...);
  };
}
```

The returned function closes over timerId. This closure is what makes repeated cancellation of the previous timer possible.

12. Throttling vs Debouncing

Property	Throttling	Debouncing
Main purpose	Limit execution frequency	Wait until events stop
Continuous events	Executes periodically	Keeps postponing execution
Timer reset	Not required in timestamp version	Required
Typical example	Scroll tracking	Search suggestions
Question answered	How often can it run?	When has the activity stopped?

13. Practical Use Cases

Throttling

- Scroll position tracking
- Infinite scrolling checks
- Window resize handling
- Mousemove processing
- Drag events
- Progress indicators
- Continuous UI updates where periodic updates are sufficient

Debouncing

- Search suggestions/autocomplete
- Search API requests
- Validation after typing stops

- Autosave after the user stops editing
- Resize calculations when only the final size matters
- Preventing repeated actions caused by rapid input

14. Throttling with setTimeout

Throttling can also be implemented with a boolean flag and setTimeout().

```
function throttle(callback, delay) {
  let canExecute = true;

  return function (...args) {
    if (!canExecute) {
      return;
    }

    callback(...args);
    canExecute = false;

    setTimeout(() => {
      canExecute = true;
    }, delay);
  };
}
```

Here, the first permitted event executes immediately. Further events are ignored until the timeout makes canExecute true again.

15. Forwarding Arbitrary Arguments

Using rest and spread syntax makes the utility functions independent of the number of callback arguments.

```
function debounce(callback, delay) {
  let timerId;

  return function (...args) {
    clearTimeout(timerId);

    timerId = setTimeout(() => {
      callback(...args);
    }, delay);
  };
}

function throttle(callback, delay) {
  let time = 0;

  return function (...args) {
    let currentTime = Date.now();

    if (currentTime - time >= delay) {
      callback(...args);
      time = currentTime;
    }
  };
}
```

16. Leading and Trailing Execution

Leading execution means executing at the beginning of an event burst. **Trailing execution** means executing after the event burst ends.

Advanced throttle/debounce implementations may provide options to control whether the leading call, trailing call, or both are performed.

```
Leading:  
Event → Callback immediately → subsequent events controlled  
  
Trailing:  
Event → Event → Event → silence → Callback
```

17. Cancellation

A production-quality debounce utility may expose a cancellation method so that pending work can be cancelled explicitly, for example when a component is destroyed.

```
function debounce(callback, delay) {  
  let timerId;  
  
  function handler(...args) {  
    clearTimeout(timerId);  
  
    timerId = setTimeout(() => {  
      callback(...args);  
    }, delay);  
  }  
  
  handler.cancel = function () {  
    clearTimeout(timerId);  
  };  
  
  return handler;  
}
```

This pattern is especially useful in applications with component lifecycles or when pending asynchronous work should no longer occur.

18. Important Details About setTimeout

- `setTimeout()` does not guarantee execution exactly after the specified delay.
- The delay is a minimum waiting period before the callback can become eligible to run.
- The callback still depends on the JavaScript runtime and event loop being able to process it.
- Calling `clearTimeout()` prevents a scheduled timeout callback from executing if it has not already run.

19. Date.now()

The timestamp-based throttle can use `Date.now()` instead of `new Date().getTime()`.

```
let currentTime = Date.now();
```

Both produce the current time in milliseconds since the Unix epoch. `Date.now()` is shorter and commonly used for this purpose.

20. Common Mistakes

- Calling the callback immediately instead of returning a wrapper function.
- Forgetting to store the returned function before registering it as an event handler.
- For debouncing, forgetting `clearTimeout()`.
- Using a single global timer for unrelated debounce instances.

- Not forwarding the event object when the callback needs it.
- Confusing throttling (periodic execution) with debouncing (execution after silence).
- Assuming setTimeout provides exact timing.

21. Advanced Concept: Separate State per Instance

Every invocation of throttle() or debounce() creates separate state through a separate closure.

```
let scrollHandler = throttle(handleScroll, 1000);
let resizeHandler = throttle(handleResize, 1000);
```

The two handlers have different internal time variables. Similarly, two debounce instances have different timerId variables.

22. Interview Questions

Q: What is throttling?

A: A technique that limits a function to execute at most once within a specified time interval.

Q: What is debouncing?

A: A technique that delays execution until a specified period has passed without another event.

Q: Main difference?

A: Throttling permits periodic execution during continuous events; debouncing waits until the event activity stops.

Q: Why is clearTimeout used in debounce?

A: To cancel the previous scheduled callback before creating a new timer.

Q: Why is closure useful here?

A: It preserves state such as timerId or the last execution timestamp between calls.

Q: Why return a function from throttle/debounce?

A: The returned wrapper function is registered as the event handler and controls when the callback executes.

Q: Why use Date.now()?

A: It provides a convenient timestamp in milliseconds for time-based throttling.

Q: Does setTimeout execute exactly after the delay?

A: No. The delay is a minimum waiting period; actual execution depends on the runtime/event loop.

Q: When use throttle?

A: When periodic updates are useful during continuous activity, such as scroll tracking.

Q: When use debounce?

A: When only the final state after the user stops triggering the event matters, such as search suggestions.

23. Mental Model

Throttle: "Keep executing, but not too frequently."

Debounce: "Keep waiting until the events stop."

```
THROTTLE
Event Event Event Event Event Event
```

↓
✓ ✓ ✓
|-----|-----|
delay delay

DEBOUNCE
Event Event Event Event
↓ ↓ ↓
timer reset reset
↓
no more events
↓
delay
↓
callback

24. Summary

- High-frequency events can cause unnecessary function executions.
- Throttling limits execution frequency.
- Debouncing delays execution until events stop for a specified period.
- Timestamp-based throttling commonly stores the last execution time.
- Debouncing commonly stores a timeout ID and cancels the previous timeout.
- Closures preserve throttle/debounce state between event-handler calls.
- `Date.now()` provides timestamps in milliseconds.
- `setTimeout` schedules a callback after a minimum delay.
- Advanced implementations may support leading/trailing execution and cancellation.
- Choose throttling when periodic updates are needed; choose debouncing when only the final state matters.